Informe de seguridad móvil — AgroSmart

Asignatura: Programación Móvil
Proyecto: AgroSmart — Flutter, Android y API NestJS
Fecha de revisión: 12 de noviembre de 2026
Integrantes: completar por el grupo
Entrega del taller: 16 de noviembre de 2026

1. Resumen ejecutivo

La revisión estática encontró riesgos en el transporte HTTP, el almacenamiento local, la configuración de producción y el envío del token de sesión. Se implementaron cambios en la app móvil para endurecer esas áreas. La app conserva HTTP para el perfil debug local; las versiones release lo bloquean y requieren HTTPS.

Esta es una revisión de código, no una certificación. Aún faltan pruebas con APK en un dispositivo, revisión dinámica del tráfico, análisis de vulnerabilidades de dependencias y una comprobación funcional de cada corrección con el backend. No se usaron cuentas, tokens ni datos reales en este informe.

2. Modelo de amenazas

Diagrama de componentes y flujo de datos

```
Usuario
  %  credenciales, datos agrícolas y mensajes
  %¼
App Flutter % % % % % % %  HTTPS (release) % % % % % % % %º API NestJS
  %                                                       %
  % %  Android Keystore / iOS Keychain                    % %  JWT, roles y finca activa
  %   sesión, caché, cola y metadatos                     % %  PostgreSQL
  %                                                       % %  servicios externos (p. ej. fotos)
  % %  Archivos offline privados (pendiente cifrar)
  % %  SMS / audio (permisos del dispositivo)

Límite de confianza: dispositivo y app no se consideran autoridad;
el servidor debe validar identidad, permisos, finca y cada dato recibido.
```

Activos, superficies y amenazas

Activo	Superficie	Amenaza identificada
JWT y sesión	Almacenamiento local, solicitudes HTTP	Lectura local del token o captura del tráfico
Datos de finca, nómina y perfil	Caché y cola offline	Acceso a datos del sandbox o copia de seguridad
Fotos y audios pendientes	Archivos locales	Extracción de archivos de un dispositivo comprometido
API	API URL configurable y red	Envío accidental de información por HTTP
Funciones por rol/finca	API REST y Socket.IO	Acceso a datos de otra finca o rol
Permisos SMS y micrófono	Manifiesto y runtime	Recolección excesiva o permisos concedidos sin necesidad

Amenazas realistas: (1) una persona con acceso al teléfono intenta extraer una sesión o datos offline; (2) alguien en la misma red intercepta una conexión HTTP; (3) un usuario autenticado intenta operar sobre otra finca o con un rol que no le corresponde.

3. Método y alcance

Se revisaron AndroidManifest.xml, la configuración Gradle de Android, los servicios Flutter de autenticación, API, caché, cola offline y adjuntos, además de referencias a permisos y almacenamiento. También se inspeccionaron el guard de autenticación y partes de la configuración NestJS para verificar los límites de confianza del servidor.

El proyecto tiene un guard global que exige JWT, valida la sesión activa y aplica roles y finca activa. Hay ValidationPipe en varios controladores, pero no se verificó que cada ruta use DTO y validación. El backend debe confirmarlo con pruebas por rol/finca.

La consulta flutter pub outdated encontró 12 dependencias bloqueadas en versiones

anteriores a las resolubles y 26 dependencias restringidas a versiones anteriores a una versión resoluble. Este comando informa versiones, no confirma ni descarta CVE.

No se pudo completar desde esta revisión: escaneo de historial Git (esta carpeta no está inicializada como repositorio), MobSF sobre un APK de release, captura de Logcat, interceptación autorizada del tráfico, revisión de certificado HTTPS del servidor ni pruebas en un teléfono. No se debe publicar información sensible en capturas.

4. OWASP Mobile Top 10 (2024)

ID | Riesgo | Estado | Evidencia y observación
M1 | Improper Credential Usage | Parcial | El cliente API ya adjunta el token en formato Bearer. No se identificó un secreto de servicio que deba ir dentro del APK; falta escanear el historial del repositorio.
M2 | Inadequate Supply Chain Security | Pendiente | Se agregó flutter_secure_storage y se resolvieron dependencias. La revisión de versiones no equivale a un escaneo CVE; falta ejecutar el análisis de dependencias exigido por el curso.
M3 | Insecure Authentication/Authorization | Parcial | El backend cuenta con guard global de JWT, roles y finca activa. Falta probar rutas con usuarios de cada rol y comprobar el aislamiento entre fincas.
M4 | Insufficient Input/Output Validation | Parcial | Varios controladores aplican ValidationPipe con whitelist y transform; la validación no está demostrada para todas las rutas.
M5 | Insecure Communication | Corregido en release | HTTP queda permitido solo en debug; el manifiesto y el cliente de red bloquean HTTP en release. Se necesita probar con el dominio HTTPS real.
M6 | Inadequate Privacy Controls | Parcial | SMS, telefonía y audio corresponden a funciones existentes. El grupo debe confirmar que cada permiso se solicita solo al usar esa función y explicar el dato recogido.
M7 | Insufficient Binary Protections | Parcial | release activa R8 y reducción de recursos y ya no usa la firma debug. La firma real está pendiente de una clave del equipo.
M8 | Security Misconfiguration | Corregido en código | Se desactivaron backups Android y el tráfico claro en release. El receptor de SMS exportado está restringido al permiso del sistema.
M9 | Insecure Data Storage | Parcial | Sesión, logins offline, caché, cola y metadatos pasan a almacenamiento seguro. Los archivos de fotos/audio pendientes aún no están cifrados individualmente.
M10 | Insufficient Cryptography | Pendiente | El verificador offline de contraseña usa SHA-256 iterado 5.000 veces. Debe migrarse a una función de derivación de contraseña (PBKDF2/Argon2) y probar el flujo offline.

5. Hallazgos y correcciones

ID | Riesgo OWASP | Severidad | Hallazgo y corrección | Estado
H-01 | M5 | Alta | La app permitía HTTP globalmente y usaba HTTP por defecto. Se limita el tráfico claro a debug, se exige HTTPS en release y se rechazan solicitudes HTTP de producción. | Implementado; falta prueba contra el servidor HTTPS real.
H-02 | M9 | Alta | Sesión/JWT, caché, cola offline y metadatos se guardaban en preferencias sin cifrar. Se migraron a flutter_secure_storage y se eliminan las copias anteriores tras confirmar la escritura segura. | Implementado; verificar actualización en un dispositivo con datos de prueba.
H-03 | M7 | Alta | El tipo release estaba firmado con la clave debug y no activaba R8. Se habilitaron minificación y reducción de recursos; el build release exige una clave propia local. | Implementado; requiere keystore del grupo para compilar.
H-04 | M1/M3 | Media | El interceptor sustituía el token por un marcador, por lo que las solicitudes autenticadas no enviaban el JWT. Se corrigió el encabezado Bearer. | Implementado; confirmar login y rutas protegidas.
H-05 | M8/M9 | Media | Android podía incluir datos de la app en copias de seguridad. Se desactivó allowBackup. | Implementado; verificar manifiesto fusionado del APK.
H-06 | M9 | Media | Las fotos y audios pendientes permanecen como archivos en el directorio privado de la app sin cifrado individual. | Pendiente; no guardar datos reales en dispositivos compartidos/comprometidos.
H-07 | M10 | Media | El hash offline usa SHA-256 iterado 5.000 veces, no una función de derivación para contraseñas. | Pendiente; cambiar el formato preservando la migración de cuentas offline.

6. Verificación antes/después

Pruebas que puede ejecutar el grupo

1. Compilar y analizar Flutter: desde AgroSmart_Fronted_Movil, ejecutar flutter analyze y flutter test. Revisar errores nuevos y registrar el resultado.
2. Probar migración: en un teléfono/emulador con una cuenta ficticia ya iniciada, instalar la versión actualizada encima de la anterior. Comprobar que conserva la sesión y la cola offline. No borrar datos de la app antes de esta prueba.
3. Probar comunicaciones: en debug, demostrar la conexión local con adb reverse. En release, probar solo con https:// y un certificado válido. No usar interceptación contra sistemas de terceros; usar únicamente el backend propio y datos ficticios.
4. Probar roles y finca: con cuentas de prueba, confirmar que un empleado no puede ejecutar acciones de administrador y que una cuenta de una finca no lee otra.
5. Revisar el APK: generar el release firmado por el grupo y revisar permisos, usesCleartextTraffic, backups y ofuscación con la herramienta permitida por el curso.
6. Capturar evidencia: adjuntar una captura del manifiesto/configuración antes y

después, los resultados de build/análisis y una demostración de dos hallazgos corregidos. Ocultar correos, teléfonos, tokens, contraseñas, dominios privados y datos personales.

Guion sugerido para la sustentación (10 minutos)

- 1 min: explicar la app, activos y límites de confianza.
- 2 min: mostrar el flujo de sesión y almacenamiento seguro.
- 3 min: demostrar HTTPS en release y el comportamiento HTTP de debug local.
- 2 min: mostrar el análisis de dos hallazgos antes/después.
- 2 min: explicar riesgos pendientes y la validación de roles/finca en el backend.

7. Verificación ejecutada

- dart analyze sobre los archivos Dart modificados: sin problemas.
- flutter test --reporter compact: 78 pruebas aprobadas.
- flutter build apk --debug: APK generado correctamente.
- flutter analyze completo: sin errores en los archivos modificados, pero el proyecto aún informa 30 advertencias/avisos en archivos ajenos a este cambio.
- flutter build apk --release --dart-define=API_URL=https://...: bloqueado intencionalmente porque aún no existe android/key.properties; el grupo debe aportar la clave de firma.
- flutter pub outdated: encontró dependencias para actualizar; falta el análisis CVE.

8. Pasos que debe completar el grupo

1. Crear un keystore Android propio y guardar los valores en AgroSmart_Fronted_Movil/android/key.properties. El ejemplo está en el README de la app. No enviar el keystore, contraseñas ni tokens al chat, al repositorio ni al PDF.
2. Configurar API_URL con el dominio HTTPS real al generar el APK release.
3. Ejecutar las pruebas de la sección anterior con cuentas y datos ficticios; registrar resultados y adjuntar capturas saneadas.
4. Escanear dependencias e historial en el repositorio del grupo y agregar los resultados verificables al informe.
5. Cifrar los archivos offline pendientes y reemplazar el hash offline por PBKDF2/Argon2 antes de considerar M9/M10 completamente cerrados.
6. Este workspace no es un repositorio Git, por lo que no se generaron commits/PR. Cuando el grupo tenga el repositorio configurado, debe revisar y registrar cada cambio con los IDs H-01 a H-05.

9. Resultado

Se implementaron cinco correcciones de código (H-01 a H-05) y se prepararon instrucciones para build de producción y pruebas. El cierre del taller depende de completar la firma propia, las pruebas dinámicas y la evidencia del grupo. No se debe afirmar que el proyecto está completamente seguro hasta resolver H-06/H-07 y verificar el backend con pruebas.